48. Embeddings, Vector Search, Similarity

48. Embeddings, Vector Search, Similarity

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `48-embeddings-vectors`

Learning objectives

- Compute cosine similarity
- Normalize vectors
- Sketch top-K retrieval
- Understand ANN vs exact search

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Embeddings power semantic search. Cosine similarity and vector stores are core AI infra skills in Go services.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Embeddings are float vectors representing meaning.
- Cosine similarity = dot/(||a||||b||).
- Normalize vectors to make dot product = cosine.
- Indexes (overview): brute force → ANN libraries.
- Dimension mismatches must error.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-embeddings, && cd /tmp/gw-embeddings,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "math"
)

func Cosine(a, b []float64) float64 {
    var dot, na, nb float64
    for i := range a {
        dot += a[i] * b[i]
        na += a[i] * a[i]
        nb += b[i] * b[i]
    }
    return dot / (math.Sqrt(na) * math.Sqrt(nb))
}

func main() { fmt.Println(Cosine([]float64{1, 0}, []float64{1, 0})) }
```

Run it:

```
go run .
```

Expected output:

```
1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement Cosine; test orthogonal vectors → 0.
2. Find argmax similarity in a tiny corpus.
3. L2-normalize a vector in place.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Embeddings

Vectors representing text/images. Similarity ≈ semantic closeness.

```
func Cosine(a, b []float64) float64 {
    return Dot(a, b) / (Norm(a) * Norm(b))
}
```

Retrieval

Exact: scan all docs O(n·d). Approximate Nearest Neighbor (ANN) libraries trade recall for speed (HNSW, IVF).

Go's role

Go often owns the service: store vectors (pgvector/remote DB), retrieve top-K, pass chunks to the LLM.

Failure modes to watch while coding

- Forgetting to normalize before dot-as-cosine
- Comparing vectors from different models
- n^2 algorithms on large corpora without ANN

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- numpy cosine vs explicit loops.
- Same math; Go forces dimension checks you might skip in notebooks.
- FAISS/Python vs Go ANN bindings — know the trade.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Store the model name + dimension with vectors — embeddings are not interchangeable.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Embeddings, Vector Search, Similarity" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Forgetting to normalize before dot-as-cosine
- Comparing vectors from different models

- n² algorithms on large corpora without ANN

Exercises

Work in `exercises/48-embeddings-vectors/`. Read that folder's `README.md` before you code.

What you will build

Implement cosine/similarity helpers for the exercise.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/48-embeddings-vectors/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Cosine vs dot product?
2. What is ANN?
3. Why model versioning matters?

Further reading

- https://en.wikipedia.org/wiki/Cosine_similarity